Predicting Indoor CO2 from Environmental Sensors

Ariel Jontof-Hutter 33879982

1. Engineering Problem

Context:
Building HVAC and ventilation are mechanical engineering systems, with most Building Management Systems (BMS) running them on a fixed schedule rather than measured conditions. This ends up wasting energy and can leave air quality poorly matched to the demand of the room. Indoor CO2 is the standard indicator of ventilation adequacy, which can make occupants uncomfortable if poorly ventilated. CO2 sensors cost more and are more prone to failure than basic temperature and humidity sensors. Monash's Smart Infrastructure network already logs temperature, humidity, pressure, VOC, particulate matter (PM) and CO2 in many rooms.

Problem addressed:
Can CO2 concentration be predicted from the other, cheaper environmental measurements taken in the same room?

Why it matters:
- Fault tolerance: a model can bridge the gaps when a CO2 sensor drops out.
- Cost: If the estimate is adequate enough, future deployments need fewer dedicated CO2 sensors.
- Control: a CO2 estimate can feed ventilation decisions in rooms without a CO2 sensor.

Intended use:
The model is intended as a fallback signal during CO2 sensor outages, or for a low cost input for rooms without a CO2 sensor.

Limitations of scope:
- The model is trained per sensor and per building, so it reflects local ventilation and room geometry and would need retraining elsewhere.
- Ventilation rate and room volume affect CO2 independently of the available inputs, which puts a ceiling on the achievable accuracy.
- Only 2 of the 5 sensors in the dataset were usable and both were in spaces where CO2 barely varies.
- The result is a statistical estimate and not a direct physical measurement

2. Inputs, Outputs and Data

The data are Monash Smart Infrastructure environmental sensor records, consisting of 5 sensors from February 2024 to April 2026. Each row stores its readings as nested JSON, which was parsed into one flat table. The database time was used as a timestamp, held in UTC and converted to Melbourne local time for the hour of the day input.

The model accepts nine numeric inputs per readings being 8 sensor variables and the local hour of the day. All are standardised using the mean and standard deviation of the training period.

Input	Meaning	Units	Domain
Temperature	The temperature of the room/area	^{o}C	11.8 to 37.5
Humidity	The humidity of the room/area	%RH	17 to 84
Pressure	The air pressure of the room/area	mb	988 to 1028
VOC	Carbon based chemicals that easily evaporate into the air as gases recorded in the room/area	ppb	0 to 8051
PM1	Particles recorded in the room/area less than 1 micron in size	$\mu g/m^3$	0 to 73
PM2.5	Particles recorded in the room/area less than 2.5 micron in size	$\mu g/m^3$	0 to 105
PM4	Particles recorded in the room/area less than 4 micron in size	$\mu g/m^3$	0 to 163
PM10	Particles recorded in the room/area less than 10 micron in size	$\mu g/m^3$	0 to 190
Hour	Local hour of the day	h	0 to 23

Table 1: model inputs, with the domains being the ranges observed in the retained rows.

The acceptable domain for each was determined through analysing the data and graphing the change in each variable in time, finding the maximum and minimum outputs, where extreme values were kept only if they were physically plausible.. There were some strange values that were found whilst sifting through the data such as a temperature reading of 37°C which is quite high for a room. However, it was found through the date of the reading, which occurred on 9/1/2026, that the outside temperature reached 44 degrees in Clayton which provides some certainty that the reading was relatively accurate. Additionally, for the VOC's, there were readings above severe levels of over 2000 ppb, which crosses the severe threshold, where people should vacate the room. However, checking when these high values occur, it occurred at relatively similar times during the day (between 7 and 10 pm), which may line up with a possible cleaning routine or aerosol product that is being used in the room. As a result, both were retained in the dataset.

The model outputs one value, the predicted CO2 concentration in ppm, with an observed range of 400 to 1387 ppm. It is an estimation of the ventilation adequacy and occupancy driven air quality change, intended as a fallback for BMS ventilation decisions.

Invalid, missing and unsupported data:

- Duplicated: 136996 of 179447 rows or 76% shared a sensor ID and device timestamp with another row and were identical in every other column. Many were reinsertions of the same readings up to 224 days apart, so they were removed, keeping the first of the readings.
- Faulty sensors: three sensors had only 1 or 2 distinct readings across all their rows, which were found through their battery and PM1 readings staying frozen at the same values, and were removed. The two remaining sensors, located in the upper compactus and one unlabelled room, were retained, which were referred to as upper compactus and unknown room for the remainder of the study.
- Missing values: formaldehyde is 67.8% missing and was dropped as an input. Rows missing any PM readings (1693 rows or 3.9%) were dropped. After this only 41146 rows of data remained (13800 upper compactus, 27346 unknown room)
- Unsupported inputs: readings from other sensors, or outside the training ranges in table 1 are outside what the model was trained on, which the pipeline does not currently flag, which is a limitation for deployment of the model.

3. Computational Approach: Support Vector Regression

Support Vector Regression (SVR) fits a function that tries to predict CO2 within a certain tolerance, called the epsilon tube, around the true values. Errors lying within this tube are not penalised by a standard epsilon SVR loss function, with the model only incurring a cost for predictions falling outside the tube boundary. These points lying outside the epsilon tube become the support vectors and only these points determine the final shape of the fitted function. To capture nonlinear behaviour, SVR uses a kernel function, which measures the similarity between feature vectors. In this project the RBF kernel was used as it allows a smooth nonlinear prediction without explicitly adding polynomial columns.

This was selected after brief initial exploratory analysis and linear regression residual diagnostics indicated the relationship between the environmental variables and CO2 was not well described by a simple linear model. There was a distinguishable pattern presented in the linear regression residuals against the predicted CO2, illustrating there is an underlying issue. SVR's kernel based approach allows it to model such non linear relationships and interactions between features, which linear regression is structurally unable to represent.

One of the assumptions with this model is that an RBF kernel can represent the relationship between the inputs and CO2. Additionally, it was assumed the epsilon tube is sensible. Too wide and real CO2 variation is treated as noise, too narrow and the model fits noise. Inputs

are standardised so that no variable dominates the distance calculation because of its units, for example pressure around 1000 mb against PM near 0.

The model implements SVR with an RBF kernel, with C=1, epsilon=0.1 and gamma = 0.1 initially. C controls the sensitivity toward the error points outside the tube, a larger C penalises points outside the tube more heavily, fitting the data more tightly at greater risk of overfitting, while a smaller C tolerates more points outside the tube in exchange for a smoother solution. Epsilon sets the tube half width of the tolerance tube itself, with errors not penalised within the tube limits. Gamma controls how local the influence of an RBF support vector is, with larger gamma meaning a more flexible solution and vice versa. These were treated as starting values, with the tuned values and how they were chosen in section 6.

The model was implemented in a single Jupyter notebook using pandas, NumPy and scikit-learn. Each sensor was sorted chronologically and split 80/20 by time, with the first 80% the training data and last 20% the testing data, with a separate model fitted per sensor. The scaler was fitted on the training data only. A pooled model trained on both sensors was also checked. It was more accurate for the upper compactus and less accurate for the unknown room, so the per-sensor models were kept. Shared helper functions handle the split, scoring, timing and plotting.

Known limitations:

- Training time grows roughly with the square of the number of samples, which was measured in section 6.
- The model must keep its support vectors, so memory and prediction time grow with the training set.
- Results are sensitive to C, gamma and epsilon, and the fitted function is hard to interpret physically.

4. Alternative Solution and Comparison

The alternative solution chosen was linear regression. Linear regression predicts CO2 as a weighted sum of inputs plus an intercept, with coefficients found by minimising the sum of squared errors in the training data. It was chosen as the baseline because it is relatively simple, has no parameters to tune and is fully interpretable, so it shows whether a simple additive relationship is already sufficient. It assumes the relationship is linear and that inputs act independently. Its weakness is that each input has one fixed slope. It cannot express, for example, temperature mattering only at certain times of the day, which the hourly CO2 profile suggests may occur. It was implemented through scikit-learn on the same standardised inputs, split and per-sensor setup as the SVR.

Two constant predictors were added as sanity checks, one always predicting the training mean, as well as one always predicting the training median. A model that cannot beat these has learned nothing useful about CO2 from the inputs.

Sensor	**Model**	**RMSE (ppm)**	**MAE (ppm)**	**R^2**

Upper Compactus	Mean predictor	34	29.86	-1.369
Upper Compactus	Median predictor	26.53	22.34	-0.442
Upper Compactus	Linear Regression	34.22	27.54	-1.4
Upper Compactus	SVR (baseline)	32.26	26.37	-1.132
Upper Compactus	SVR (optimised)	26.26	22.06	-0.413
Unknown Room	Mean predictor	30.21	17.07	-0.006
Unknown Room	Median predictor	30.65	16.78	-0.035
Unknown Room	Linear regression	29.58	18.11	0.035
Unknown Room	SVR (baseline)	28.82	15.37	0.085
Unknown Room	SVR (optimised)	28.64	15.14	0.096

Table 2: test period accuracy. Lower RMSE and MAE are better, and R^2 below 0 means worse than predicting the test period mean.

In terms of accuracy, for the unknown room, SVR beats both constant predictors and linear regression, but only by about 5% in RMSE (28.64 against 30.21 ppm for the mean), with R^2 near 0.1. For the upper compactus, neither linear regression nor the baseline SVR is better than a constant, and the optimised SVR is only as good as the median predictor. Its predictions correlate with actual CO2, so its improvements come from matching the level and not tracking the signal. Upper compactus negative R^2 values arise because the mean CO2 fell from 479 ppm in training to 453 ppm in the test period, a large enough shift which no input could accurately predict.

Other criteria:

- Robustness: Linear regression is the more stable of the two, which can be seen in section 5.
- Runtime and memory: Linear regression fits in milliseconds and stores 0.5 KB. SVR takes seconds and stores 0.9 to 1.8 MB, seen in section 6.
- Implementation complexity and interpretability: Linear regression has no tuning and its coefficients can be read directly. SVR needs a three parameter search and its prediction cannot be summarised as a rule and thus is more complex.
- Suitability: neither method is accurate enough to replace a CO2 sensor in these rooms. If one must be used as a fallback, linear regression gives almost the same accuracy for a tiny fraction of the cost.

In conclusion, the evidence supports SVR as slightly more accurate for the unknown room and no better than a constant for the upper compactus. The main finding is that these inputs carry little information about CO2 in these rooms.

5. Validation

What was validated: The question is whether a model trained on a sensor's past can estimate that sensor's CO2 in a later period it has not seen. Therefore, validation uses withheld test data, plus comparison with reference predictors and a sensitivity test on where the train/test boundary falls.

Why this method: A soft sensor is used forward in time, so each sensor's record was split chronologically. A random split would be misleading because readings close to each other are strongly correlated, so test rows would have near copies in the training set. The comparison with mean and median predictors guards a model that looks only good because CO2 varies little.

Metrics and acceptance criteria: RMSE and MAE are reported in ppm, where RMSE penalises large errors more and MAE is less affected by rare spikes, along with R^2. The acceptance criterion is that a model must beat the constant predictors on the test period (R^2 above 0). It must also have an RMSE clearly below the spread of CO2 in that period, which is 22.1 ppm in the upper compactus and 30.1 ppm in the unknown room. No model meets the second criteria by a useful margin.

Split robustness: The train/test boundary was moved from 60% to 90% and the test RMSE was recomputed.

Sensor	Train fraction	SVR (baseline)	SVR (optimised)	Linear regression	Mean predictor
Upper compactus	0.6	38.45	43.42	39.28	41.05
Upper compactus	0.7	36.27	39.47	37.75	39.35
Upper compactus	0.8	32.36	26.26	34.22	34
Upper compactus	0.9	20.58	20.88	21.89	27.44
Unknown room	0.6	29.45	30.06	28.62	29.98
Unknown room	0.7	30.22	29.97	30.44	31.63
Unknown room	0.8	28.81	28.64	29.58	30.20
Unknown room	0.9	25.83	25.52	26.18	27.52

Table 3: test RMSE (ppm) as the train/test split moves

Linear regression beats the main predictor in 7 of the 8 splits. The optimised SVR for the upper compactus is only better than the baseline at the 0.8 split it was tuned on and is worse at 0.6 and 0.7. Its advantage in table 2 is therefore a result of tuning to one test period and not a general improvement.

Bias and error: splitting the error into an offset and a remainder shows the optimised upper compactus SVR has a mean error of -14.2 ppm and an error standard deviation of 22.1 ppm. The latter equals the standard deviation of the actual CO2 itself, so its predictions are

effectively constant, which can be seen in Figure 1. Residual plots against the predicted value and over time are found in figures 2-5. These plots are not random which means there may be inherent biases in the model or something else such as sensor drift.

Did validation influence model selection: Yes, which weakens the result. C, gamma and epsilon were chosen by minimising RMSE on the same test period that is reported, so the optimised SVR figures are optimistic. Cleaner practice is a separate validation period carved from the end of the training data, with the test period scored once. The 80/20 split was kept as it follows the method taught, and table 3 is used to show how much the tuning result can be trusted.

What validation cannot establish:

- Performance in other rooms and buildings, since only 2 sensors and 1 defined test block each were available.
- True uncertainty, as consecutive test readings are correlated, so the effective number of independent test points is much smaller.

6. Optimisation and Performance

Tuning the SVR: C and gamma were searched on logarithmic grids, with epsilon fixed at 0.1, and then epsilon was swept at the best C and gamma, shown in figure 6. Any best cell on a grid edge triggered an extended grid, as seen in figure 7.

Sensor	Baseline (C, gamma, epsilon)	Tuned (C, gamma, epsilon)	Baseline RMSE (ppm)	Tuned RMSE (ppm)
Upper compactus	1,0.1,0.1	0.0001,100, 0.5	32.26	26.26
Unknown room	1,0.1,0.1	10,0.1,5	28.81	28.64

Table 4: SVR parameters before and after tuning.

For the unknown room, the optimum was bracketed (C =10, gamma=0.1 lies inside the combined grid), and epsilon changed RMSE by only 0.14 ppm across 0.01 to 5. For the upper compactus, the search ran to the edge of both extended grids and the optimum is a largely unusable model, with C=0.0001 and gamma=100. The kernel between distinct standardised points is almost 0 and the penalty is tiny, so the fit collapses to roughly its intercept. Epsilon changed its RMSE by only 0.27 ppm. This flat landscape means the available inputs are the limit on accuracy.

Sensor	Model	Fit (s)	Predict (s)	Support vectors	Stored size (KB)
Upper compactus	Linear regression	0.0016	0.0001	N/A	0.55
Upper compactus	SVR (baseline)	3.18	1.47	11007	904

Upper compactus	SVR (tuned)	3.76	1.70	10870	893
Unknown room	Linear regression	0.0028	0.0001	N/A	0.55
Unknown room	SVR (baseline)	14.25	5.85	21716	1782
Unknown room	SVR (tuned)	10.1	4.03	14847	1219

Table 5: cost of each model, with the prediction covering 2760 rows for the upper compactus and 5470 rows for the unknown room, with for runtime and memory, times are medians of repeated runs

At baseline the SVR keeps 99% of the training points as support vectors, because epsilon = 0.1 ppm is far narrower than the sensor's 1 ppm resolution. The wider tube chosen for the unknown room (epsilon =5) keeps 68% of the data, which cuts fit time by 29%, prediction by 31% and the model size by 32% with no loss of accuracy. This is the one clear positive outcome that resulted from tuning.

Scaling: Training on 10% to 100% of each sensor's training rows gave fit time proportional to n^1.97 to n^2.03 for the SVR model and n^0.37 to n^0.48 for linear regression. This means that doubling the data would roughly quadruple the SVR training time.

Operation counts and arithmetic intensity: One RBF kernel evaluation over p = 9 inputs costs about 3p + 3 = 30 flops and reads two 9-element float64 vectors (144 bytes), coming to an arithmetic intensity of approximately 0.21 flops/byte. This is relatively low, so the method is more likely memory bound rather than compute bound.

Dividing predict flops by the measured baseline prediction times gives about 0.6 GFLOPs/s for both sensors, which is well below what the processor can do. SVR needs roughly 1000 to 40000 times more calculations than linear regression.

Final design: Runtime is not a barrier for this use with sensors reporting about every 10 minutes, and one prediction taking under a millisecond, therefore accuracy and robustness will make the final decision. Table 3 supports the tuned SVR for the unknown room, as the results were better than the mean in three of the four splits. For the upper compactus, no model reliably beats a constant, so linear regression is the better choice if an estimation is required. Both choices are made with the caveat in section 5 that tuning used the test period.

7. Limitations and Lessons Learnt

Main finding: Temperature, humidity, pressure, VOC, particulate matter and hour of the day carry little information about CO2 in these 2 rooms. The best model, SVR for the unknown room reduces RMSE by about 5% against the predicting mean. Physically this is plausible as the two rooms could have low occupancy and could be ventilated, so the CO2 stays near the same constant value of 450 ppm, and ventilation rate and occupancy, which drive the

CO2 levels, could not be linked due to the ID's being invalid, as well as not being measured. A BMS could not currently use this estimate as a sensor replacement.

Limitations:

- Only 2 of the 5 sensors survived the data audit and neither could be reliably linked to a room function, so findings cannot be generalised to occupied teaching spaces.
- Hyperparameters were tuned on the test period and a single chronological split was used for the main results.
- Each test period is one contiguous block of correlated readings, so the accuracy estimates are uncertain.

Lessons:

- Audit the data before modelling. 76% of the rows of data were reinserted duplicates and 3 of the 5 sensors were frozen. Evidence for this came from independent signals (the constant battery voltage and the frozen PM1), not only from the duplicate count. Modelling the raw data would have given misleading results.
- Always include a constant baseline. The negative R^2 values and the 19% RMSE improvement looked meaningful until compared with the median predictor and the standard deviation of the target.
- Check timestamps and time zones. Using device time, UTC and local time interchangeably produced wrong hourly patterns until corrected.
- A flat tuning landscape is a finding. When the search ran to the edge of the grid and RMSE barely changed, it signalled a data limit and further tuning would only have overfit the test period.

Proposed Improvements:

- Tune on separate validation periods and score the test period once or use blocked time series cross validation.
- Try decision tree regression or a neural network which may handle the spiky VOC and PM inputs better and compare on the same split.

8. AI-use and Reflection

Tools and level of contribution: Claude was used throughout the project, mainly for the coding segment of the assignment. Roughly 90% of the code was written by AI, with the other 10% coming from slight editing and commenting done by me. The report was written 90% by me and the other 10% coming from AI fixing the language of the report.

What is was used or and why:

- Scoping: comparing different project ideas and datasets and the rough complexity of each, as well as explaining what a soft sensing problem is.
- Code: drafting the JSON parser, cleaning steps, plotting, SVR grid search, timing and profiling cells, and explaining any of the code that I didn't understand. I used it because I was new to scikit-learn, git and Jupyter so it could provide the code needed to get the results I was asking for.

- Debugging: reading tracebacks, as well as setup issues with VS code, github and the python environments
- Report: helping me reword what I was trying to convey in a more concise manner

How AI output was checked: I ran every cell and inspected the outputs and plots using my own judgement to see if the output results line up. I checked the AI's data claims against the raw files, for example opening the raw data to confirm that no device_time input existed in some records. There were some instances of the results seeming a little off such as the temperature readings, for which I checked these extreme temperature readings against the weather record for that day. Additionally, it seemed very off that so much of the data was unusable due to duplicate readings, missing rows, etc and plotting battery voltage and PM1 across the sensors was done to test if this was the case.

Decisions that remained mine: Choosing the CO2 soft sensing problem after pivoting from the occupancy datasets, using SVR as the main method and linear regression as the alternative, dropping the decision tree model as well as keeping the 80/20 data split, keeping the heatwave and the VOC extremes, which sensors to include, handling the missing/invalid/duplicate data as well as how I wanted the output figures to look.

AI errors and unhelpful suggestions:

- The first de-duplication step compared all the columns, including the database timestamp, so it found 0 duplicates and the later fully identical check had the same flaw.
- The AI treated the repeated rows as duplicates before it had proof, which I had to question whether the identical values could be genuine readings, which the battery voltage and PM1 checks were used to test if this was the case.
- Hourly plots first used UTC hours which hid the daily pattern.
- The AI had me tune the parameters on the same test split I reported on, which made the results seem optimistic. This was spotted on review and I chose to keep the 80/20 split and report as a limitation.
- The AI spat out code many times that I did not ask for such as plotting the residual distribution, which I kept as it provides useful insight.
- The AI suggested to keep going lower for the tuning of the C parameter, which was already at a low 0.01, which it failed to interpret how impractical the values were already.

Effect on my understanding and workflow: AI sped up the setup and let me test more ideas, but several of its errors were silent and were only caught because I questioned numbers that looked wrong. I now treat a plausible output as unverified until I have a baseline and a plot, where I can check the results and give my own judgment to determine its accuracy. I also understand SVR parameters, time series splitting and data leakage far better than when I started. Additionally, it sped the workflow up drastically, as my coding ability is not profound enough to be able to code to this ability at this rate.

9. References

- *Clayton, Victoria, Australia monthly weather | AccuWeather* (2026) *https://twitter.com/accuweather*. Available at: https://www.accuweather.com/en/au/clayton/15846/january-weather/15846 (Accessed: October 12, 2026). *Used to check the weather on 9 January 2026.*
- *Monash University Smart Infrastructure environment sensor dataset, which was given by the unit.*

10. Appendix

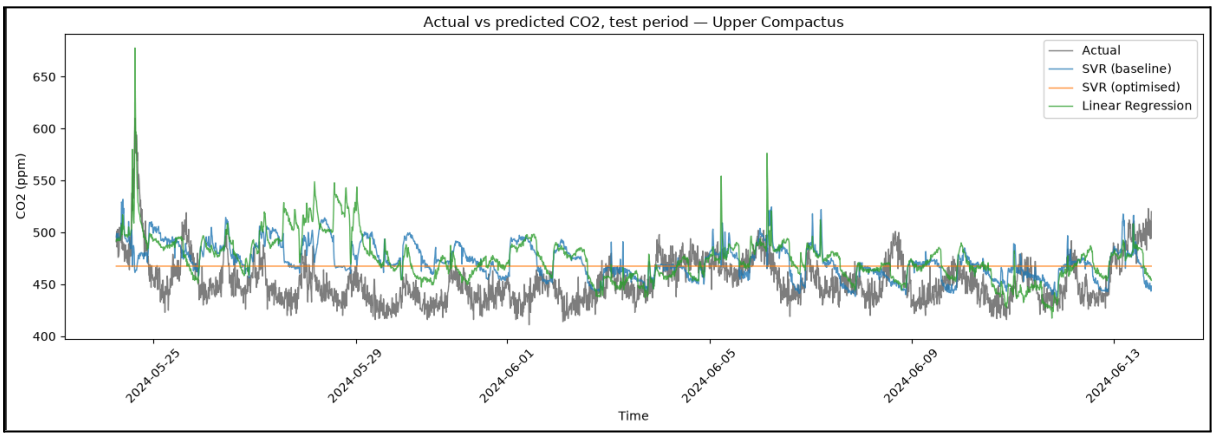


Figure 1: the actual vs predicted CO2 in the test period, highlighting the almost constant fit.

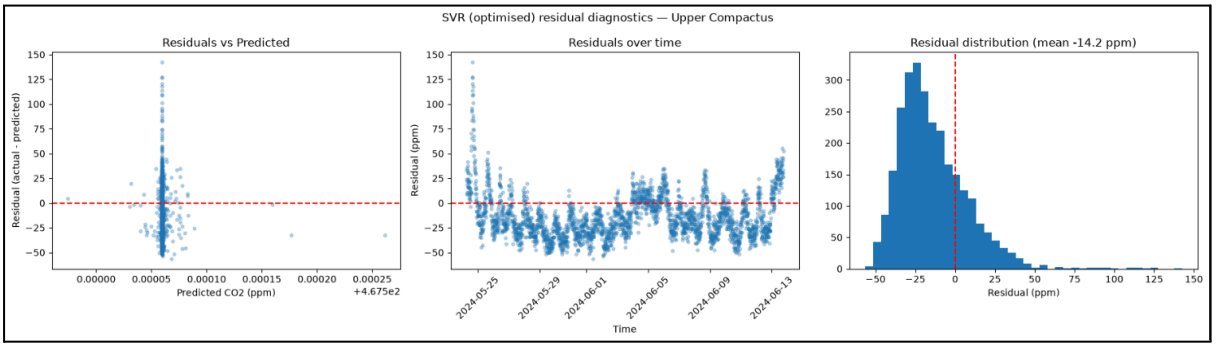


Figure 2: residual plots for the optimised SVR for the upper compactus sensor, illustrating an underlying pattern in the errors, possibly due to bias.

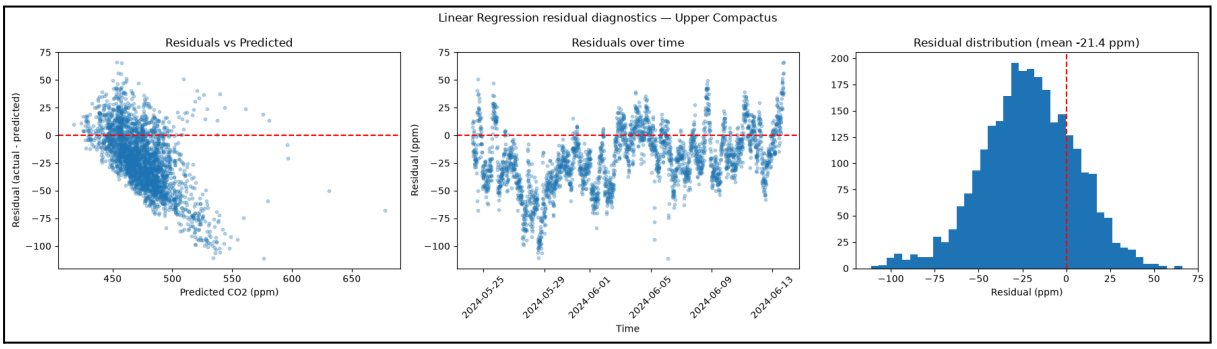


Figure 3: residual plots for the linear regression model for the upper compactus sensor, further showing an underlying pattern

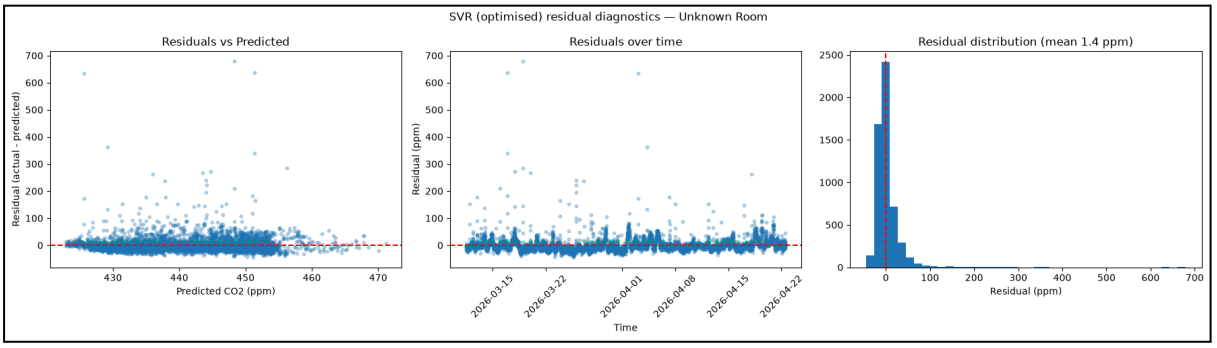


Figure 4: residual plots for the optimised SVR for the unknown room, which is still not randomly scattered, further exhibiting an issue with the modelling.

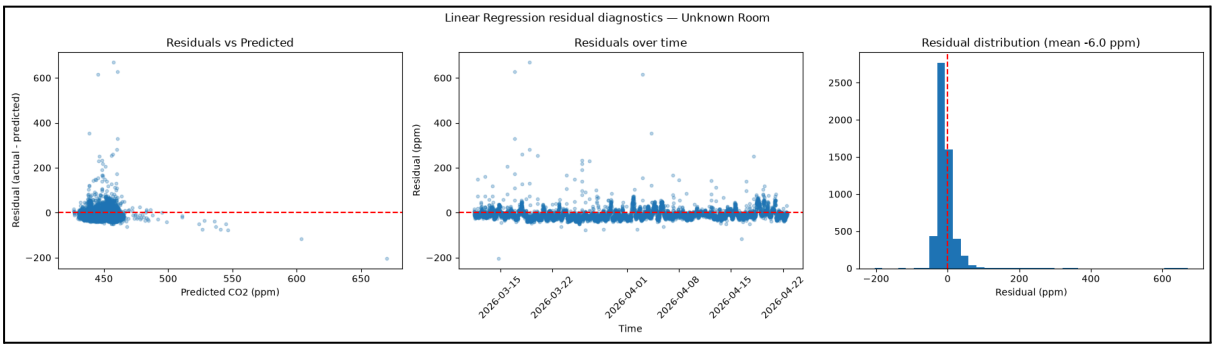


Figure 5: residual plot for the linear regression model for the unknown room sensor, displaying issues with the modelling.

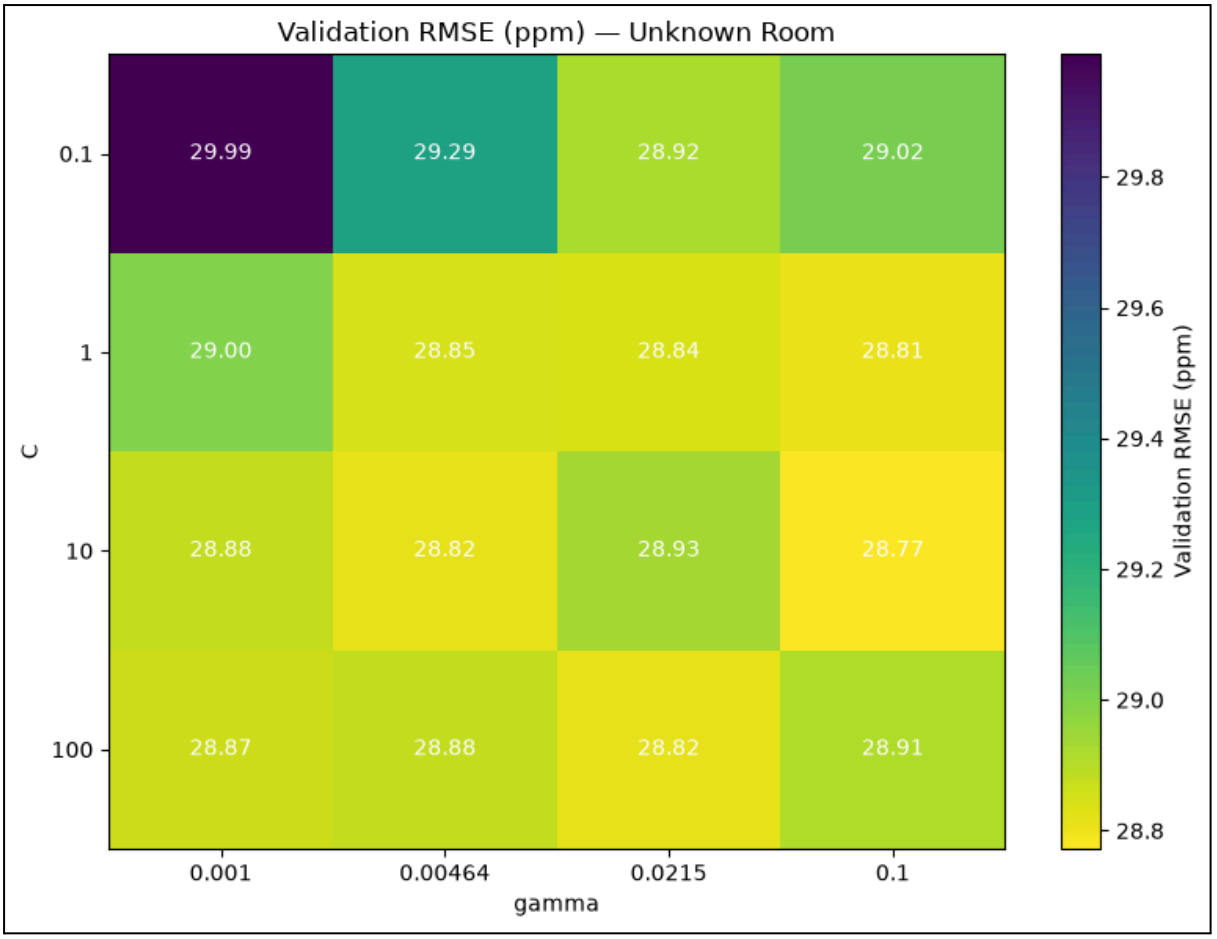


Figure 6: Heatmap of the validation RMSE for the SVR parameters, used for tuning the SVR model to find the optimal parameters

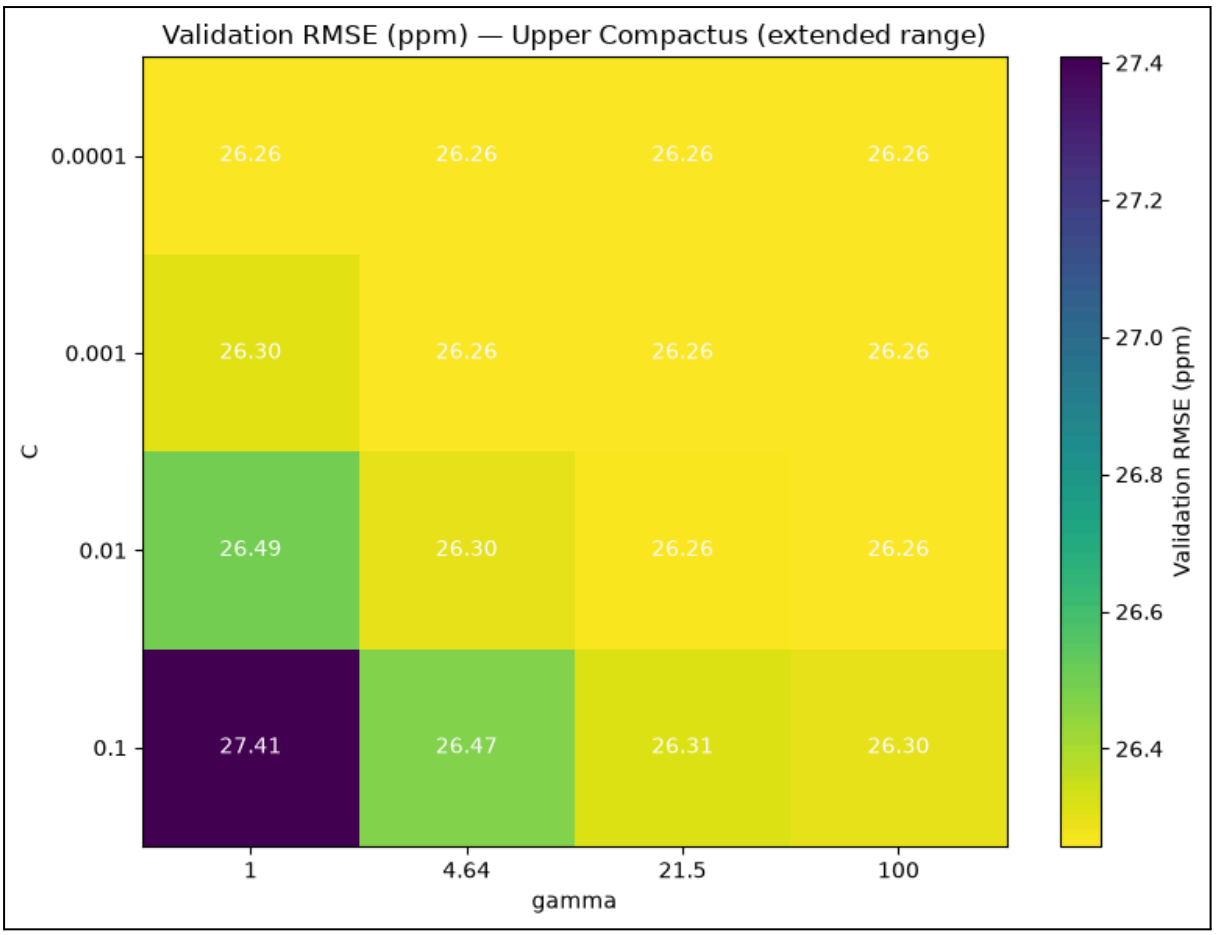


Figure 7: The heatmap of the validation RMSE for the extended range of the SVR parameters for the upper compactus.